管仲模拟器代码流程

config.py文件

代码块

```
import os
from enum import Enum

class Difficulty(Enum):
    EASY = {"income_multiplier": 1.2, "stability_impact": 0.8}
    NORMAL = {"income_multiplier": 1.0, "stability_impact": 1.0}
    HARD = {"income_multiplier": 0.8, "stability_impact": 1.2}

class Config:
    # 请替换为你自己的API密钥
    DOUBAO_API_KEY = os.getenv("DOUBAO_API_KEY", "573ab249-3130-4926-be5f-da98998863ef")
    DOUBAO_API_URL = os.getenv("DOUBAO_API_URL", "https://ark.cn-beijing.volces.com/api/v3/chat/completions")
    ECONOMY_SETTINGS = {
        "base_rates": {
            "tax_income": 0.05,
            "grain_yield": 0.15
        },
        "difficulty": Difficulty.NORMAL
    }
    HISTORICAL_EVENTS = {
        "enable": True,
        "frequency": 0.3  # 30%概率每年触发事件
    }
```

economy.py文件

代码块

```
from flask_sqlalchemy import SQLAlchemy
from datetime import datetime

db = SQLAlchemy()

class EconomyModel(db.Model):
    id = db.Column(db.Integer, primary_key=True)
```

```
    name = db.Column(db.String(100), nullable=False)
    description = db.Column(db.Text)
    category = db.Column(db.String(50))  # 经济政策、思想等
    created_at = db.Column(db.DateTime, default=datetime.utcnow)

    def to_dict(self):
        return {
            'id': self.id,
            'name': self.name,
            'description': self.description,
            'category': self.category,
            'created_at': self.created_at.isoformat()
        }
```

user.py文件

代码块

```
from flask_sqlalchemy import SQLAlchemy
from datetime import datetime

db = SQLAlchemy()

class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(80), unique=True, nullable=False)
    password = db.Column(db.String(120), nullable=False)
    created_at = db.Column(db.DateTime, default=datetime.utcnow)

    def to_dict(self):
        return {
            'id': self.id,
            'username': self.username,
            'created_at': self.created_at.isoformat()
        }
```

auth.py文件

代码块

```
from flask import Blueprint, request, jsonify
from flask_jwt_extended import create_access_token
from backend.models.user import User, db

auth_bp = Blueprint('auth', __name__)

```

```python
@auth_bp.route('/api/auth/register', methods=['POST'])
def register():
    """用户注册"""
    data = request.get_json()
    username = data.get('username')
    password = data.get('password')

    if not username or not password:
        return jsonify({"message": "用户名和密码不能为空"}), 400

    if User.query.filter_by(username=username).first():
        return jsonify({"message": "用户名已存在"}), 400

    user = User(username=username, password=password)
    db.session.add(user)
    db.session.commit()

    return jsonify({"message": "注册成功"}), 201

@auth_bp.route('/api/auth/login', methods=['POST'])
def login():
    """用户登录"""
    data = request.get_json()
    username = data.get('username')
    password = data.get('password')

    user = User.query.filter_by(username=username, password=password).first()

    if user:
        access_token = create_access_token(identity=user.id)
        return jsonify({
            "access_token": access_token,
            "user_id": user.id,
            "username": user.username
        }), 200
    else:
        return jsonify({"message": "用户名或密码错误"}), 401

@auth_bp.route('/api/auth/user', methods=['GET'])
def get_user_info():
    """获取用户信息（示例）"""
    return jsonify({
        "message": "用户信息接口",
        "endpoint": "/api/auth/user"
    })
```

dialogue.py文件

代码块

```python
from flask import Blueprint, request, jsonify
from flask_jwt_extended import jwt_required, get_jwt_identity
from backend.models.user import db

dialogue_bp = Blueprint('dialogue', __name__)

# 临时对话存储（实际项目中使用数据库）
dialogues = {}

@dialogue_bp.route('/api/dialogues', methods=['POST'])
@jwt_required()
def create_dialogue():
    """创建新对话"""
    data = request.get_json() or {}
    user_id = get_jwt_identity()

    dialogue_id = len(dialogues) + 1
    dialogues[dialogue_id] = {
        'id': dialogue_id,
        'user_id': user_id,
        'title': data.get('title', '新对话'),
        'messages': [],
        'created_at': '2025-01-20T10:00:00'
    }

    return jsonify(dialogues[dialogue_id]), 201

@dialogue_bp.route('/api/dialogues', methods=['GET'])
@jwt_required()
def get_dialogues():
    """获取对话列表"""
    user_id = get_jwt_identity()
    user_dialogues = [d for d in dialogues.values() if d['user_id'] == user_id]

    return jsonify({
        'dialogues': user_dialogues,
        'total': len(user_dialogues)
    }), 200

@dialogue_bp.route('/api/dialogues/<int:dialogue_id>', methods=['GET'])
@jwt_required()
def get_dialogue(dialogue_id):
    """获取对话详情"""
```

```python
    dialogue = dialogues.get(dialogue_id)
    if not dialogue:
        return jsonify({"message": "对话不存在"}), 404

    user_id = get_jwt_identity()
    if dialogue['user_id'] != user_id:
        return jsonify({"message": "无权访问此对话"}), 403

    return jsonify(dialogue), 200

@dialogue_bp.route('/api/dialogues/<int:dialogue_id>/messages', methods=['POST'])
@jwt_required()
def add_message(dialogue_id):
    """添加消息到对话"""
    data = request.get_json()
    message_content = data.get('message')

    if not message_content:
        return jsonify({"message": "消息内容不能为空"}), 400

    dialogue = dialogues.get(dialogue_id)
    if not dialogue:
        return jsonify({"message": "对话不存在"}), 404

    user_id = get_jwt_identity()
    if dialogue['user_id'] != user_id:
        return jsonify({"message": "无权访问此对话"}), 403

    # 添加用户消息
    dialogue['messages'].append({
        'sender': 'user',
        'content': message_content,
        'timestamp': '2025-01-20T10:00:00'
    })

    # 模拟AI回复
    dialogue['messages'].append({
        'sender': 'guanzhong',
        'content': f"收到您的消息: {message_content}。管仲正在思考中...",
        'timestamp': '2025-01-20T10:01:00'
    })

    return jsonify({
        'message': '消息添加成功',
        'dialogue_id': dialogue_id
    }), 200
```

economy.py文件

代码块

```python
from flask_sqlalchemy import SQLAlchemy
from datetime import datetime

db = SQLAlchemy()

class EconomyModel(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(100), nullable=False)
    description = db.Column(db.Text)
    category = db.Column(db.String(50))  # 经济政策、思想等
    created_at = db.Column(db.DateTime, default=datetime.utcnow)

    def to_dict(self):
        return {
            'id': self.id,
            'name': self.name,
            'description': self.description,
            'category': self.category,
            'created_at': self.created_at.isoformat()
        }
```

user.py文件

代码块

```python
from flask_sqlalchemy import SQLAlchemy
from datetime import datetime

db = SQLAlchemy()

class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(80), unique=True, nullable=False)
    password = db.Column(db.String(120), nullable=False)
    created_at = db.Column(db.DateTime, default=datetime.utcnow)

    def to_dict(self):
        return {
            'id': self.id,
            'username': self.username,
            'created_at': self.created_at.isoformat()
```

```
        }
```

auth.py文件

代码块

```python
from flask import Blueprint, request, jsonify
from flask_jwt_extended import create_access_token
from backend.models.user import User, db

auth_bp = Blueprint('auth', __name__)

@auth_bp.route('/api/auth/register', methods=['POST'])
def register():
    """用户注册"""
    data = request.get_json()
    username = data.get('username')
    password = data.get('password')

    if not username or not password:
        return jsonify({"message": "用户名和密码不能为空"}), 400

    if User.query.filter_by(username=username).first():
        return jsonify({"message": "用户名已存在"}), 400

    user = User(username=username, password=password)
    db.session.add(user)
    db.session.commit()

    return jsonify({"message": "注册成功"}), 201

@auth_bp.route('/api/auth/login', methods=['POST'])
def login():
    """用户登录"""
    data = request.get_json()
    username = data.get('username')
    password = data.get('password')

    user = User.query.filter_by(username=username, password=password).first()

    if user:
        access_token = create_access_token(identity=user.id)
        return jsonify({
            "access_token": access_token,
            "user_id": user.id,
            "username": user.username
```

```
        }), 200
    else:
        return jsonify({"message": "用户名或密码错误"}), 401

@auth_bp.route('/api/auth/user', methods=['GET'])
def get_user_info():
    """获取用户信息（示例）"""
    return jsonify({
        "message": "用户信息接口",
        "endpoint": "/api/auth/user"
    })
```

dialogue.py

代码块

```
from flask import Blueprint, request, jsonify
from flask_jwt_extended import jwt_required, get_jwt_identity
from backend.models.user import db

dialogue_bp = Blueprint('dialogue', __name__)

# 临时对话存储（实际项目中使用数据库）
dialogues = {}

@dialogue_bp.route('/api/dialogues', methods=['POST'])
@jwt_required()
def create_dialogue():
    """创建新对话"""
    data = request.get_json() or {}
    user_id = get_jwt_identity()

    dialogue_id = len(dialogues) + 1
    dialogues[dialogue_id] = {
        'id': dialogue_id,
        'user_id': user_id,
        'title': data.get('title', '新对话'),
        'messages': [],
        'created_at': '2025-01-20T10:00:00'
    }

    return jsonify(dialogues[dialogue_id]), 201

@dialogue_bp.route('/api/dialogues', methods=['GET'])
@jwt_required()
def get_dialogues():
```

```python
    """获取对话列表"""
    user_id = get_jwt_identity()
    user_dialogues = [d for d in dialogues.values() if d['user_id'] == user_id]

    return jsonify({
        'dialogues': user_dialogues,
        'total': len(user_dialogues)
    }), 200

@dialogue_bp.route('/api/dialogues/<int:dialogue_id>', methods=['GET'])
@jwt_required()
def get_dialogue(dialogue_id):
    """获取对话详情"""
    dialogue = dialogues.get(dialogue_id)
    if not dialogue:
        return jsonify({"message": "对话不存在"}), 404

    user_id = get_jwt_identity()
    if dialogue['user_id'] != user_id:
        return jsonify({"message": "无权访问此对话"}), 403

    return jsonify(dialogue), 200

@dialogue_bp.route('/api/dialogues/<int:dialogue_id>/messages', methods=
['POST'])
@jwt_required()
def add_message(dialogue_id):
    """添加消息到对话"""
    data = request.get_json()
    message_content = data.get('message')

    if not message_content:
        return jsonify({"message": "消息内容不能为空"}), 400

    dialogue = dialogues.get(dialogue_id)
    if not dialogue:
        return jsonify({"message": "对话不存在"}), 404

    user_id = get_jwt_identity()
    if dialogue['user_id'] != user_id:
        return jsonify({"message": "无权访问此对话"}), 403

    # 添加用户消息
    dialogue['messages'].append({
        'sender': 'user',
        'content': message_content,
        'timestamp': '2025-01-20T10:00:00'
```

```
    })

    # 模拟AI回复
    dialogue['messages'].append({
        'sender': 'guanzhong',
        'content': f"收到您的消息: {message_content}。管仲正在思考中...",
        'timestamp': '2025-01-20T10:01:00'
    })

    return jsonify({
        'message': '消息添加成功',
        'dialogue_id': dialogue_id
    }), 200
```

economy.py

代码块

```
from flask import Blueprint, request, jsonify
from backend.models.economy import EconomyModel, db

economy_bp = Blueprint('economy', __name__)

@economy_bp.route('/api/economy/concepts', methods=['GET'])
def get_economy_concepts():
    """获取管仲经济思想概念"""
    concepts = [
        {
            "id": 1,
            "name": "相地而衰征",
            "category": "土地政策",
            "description": "根据土地优劣征收不同赋税的土地改革政策",
            "importance": 95
        },
        {
            "id": 2,
            "name": "轻重九府",
            "category": "经济调控",
            "description": "国家调控市场、平衡物价的经济管理方法",
            "importance": 90
        },
        {
            "id": 3,
            "name": "官山海",
            "category": "资源管理",
            "description": "国家垄断盐铁等重要资源的管理政策",
```

```
            "importance": 85
        }
    ]
    return jsonify(concepts), 200

@economy_bp.route('/api/economy/graph', methods=['GET'])
def get_economy_graph():
    """获取经济思想知识图谱"""
    graph_data = {
        "nodes": [
            {"id": "gz", "label": "管仲", "category": "人物", "size": 30},
            {"id": "jj", "label": "经济思想", "category": "思想", "size": 25},
            {"id": "xd", "label": "相地而衰征", "category": "政策", "size": 20},
            {"id": "qz", "label": "轻重九府", "category": "政策", "size": 20},
            {"id": "gsh", "label": "官山海", "category": "政策", "size": 20}
        ],
        "edges": [
            {"source": "gz", "target": "jj", "label": "提出"},
            {"source": "jj", "target": "xd", "label": "包含"},
            {"source": "jj", "target": "qz", "label": "包含"},
            {"source": "jj", "target": "gsh", "label": "包含"}
        ]
    }
    return jsonify(graph_data), 200

@economy_bp.route('/api/economy/simulate', methods=['POST'])
def simulate_economy():
    """经济政策模拟"""
    data = request.get_json()
    policy = data.get('policy')
    parameters = data.get('parameters', {})

    # 模拟不同政策的效果
    simulations = {
        "相地而衰征": {
            "effect": "财政收入增加20%",
            "stability": "社会稳定性提高",
            "duration": "长期有效"
        },
        "轻重九府": {
            "effect": "市场物价稳定",
            "stability": "经济波动减少",
            "duration": "中期调控"
        },
        "官山海": {
            "effect": "国家资源收入大幅增加",
            "stability": "资源分配更合理",
```

```
            "duration": "长期政策"
        }
    }

    result = simulations.get(policy, {
        "effect": "政策效果待评估",
        "stability": "未知",
        "duration": "需要更多数据"
    })

    return jsonify({
        "policy": policy,
        "parameters": parameters,
        "simulation_result": result
    }), 200
```

game.py

代码块

```
from flask import Blueprint, request, jsonify

game_bp = Blueprint('game', __name__)

@game_bp.route('/api/game/scenarios', methods=['GET'])
def get_game_scenarios():
    """获取历史场景列表"""
    scenarios = [
        {
            "id": 1,
            "title": "九合诸侯",
            "description": "体验管仲辅佐齐桓公'九合诸侯，一匡天下'的历史场景",
            "difficulty": "中等",
            "estimated_time": 45,
            "image": "/static/images/scenario1.jpg"
        },
        {
            "id": 2,
            "title": "相地而衰征",
            "description": "模拟管仲实施土地改革政策的历史场景",
            "difficulty": "简单",
            "estimated_time": 30,
            "image": "/static/images/scenario2.jpg"
        },
        {
            "id": 3,
```

```
            "title": "管鲍之交",
            "description": "重现管仲与鲍叔牙深厚友谊的历史场景",
            "difficulty": "简单",
            "estimated_time": 25,
            "image": "/static/images/scenario3.jpg"
        }
    ]
    return jsonify(scenarios), 200

@game_bp.route('/api/game/scenarios/<int:scenario_id>', methods=['GET'])
def get_scenario_detail(scenario_id):
    """获取场景详情"""
    scenarios_data = {
        1: {
            "id": 1,
            "title": "九合诸侯",
            "description": "详细描述...",
            "objectives": ["达成外交协议", "维护齐国霸权"],
            "characters": ["管仲", "齐桓公", "诸侯"],
            "duration": "45分钟"
        },
        2: {
            "id": 2,
            "title": "相地而衰征",
            "description": "详细描述...",
            "objectives": ["实施土地改革", "平衡赋税"],
            "characters": ["管仲", "土地官员", "农民"],
            "duration": "30分钟"
        },
        3: {
            "id": 3,
            "title": "管鲍之交",
            "description": "详细描述...",
            "objectives": ["建立信任关系", "展现才能"],
            "characters": ["管仲", "鲍叔牙", "齐桓公"],
            "duration": "25分钟"
        }
    }

    scenario = scenarios_data.get(scenario_id)
    if not scenario:
        return jsonify({"message": "场景不存在"}), 404

    return jsonify(scenario), 200

@game_bp.route('/api/game/start/<int:scenario_id>', methods=['POST'])
def start_scenario(scenario_id):
```

```
    """开始游戏场景"""
    return jsonify({
        "scenario_id": scenario_id,
        "status": "started",
        "message": "场景已开始，请按照指引进行操作",
        "first_step": "了解当前局势和任务目标"
    }), 200

@game_bp.route('/api/game/action', methods=['POST'])
def game_action():
    """执行游戏动作"""
    data = request.get_json()
    action = data.get('action')
    scenario_id = data.get('scenario_id')

    # 模拟动作结果
    results = {
        "diplomacy": "外交成功，诸侯信服",
        "economic_reform": "经济改革初见成效",
        "military": "军事部署完成",
        "advice": "建议被采纳"
    }

    result = results.get(action, "动作执行中...")

    return jsonify({
        "action": action,
        "result": result,
        "next_step": "请继续下一步操作",
        "score_change": +10
    }), 200
```

api_design.py

代码块

```
# 管仲AI后端API设计规范
# 版本: v1.0
# 最后更新: 2025-01-20

"""
管仲AI后端API设计文档
基于OpenAPI 3.0规范，与实际代码保持同步
"""

# ========================
```

```
# 1. 用户认证模块
# ========================

"""
POST /api/auth/register
描述: 用户注册
请求体:
{
    "username": "string, 必需",
    "password": "string, 必需"
}
响应:
- 201: {"message": "注册成功"}
- 400: {"message": "用户名已存在"}
"""

"""
POST /api/auth/login
描述: 用户登录
请求体:
{
    "username": "string, 必需",
    "password": "string, 必需"
}
响应:
- 200: {"access_token": "JWT令牌"}
- 401: {"message": "认证失败"}
"""

"""
POST /api/auth/logout
描述: 用户登出（客户端清除token）
认证: 需要Bearer Token
响应:
- 200: {"message": "登出成功"}
"""

"""
GET /api/auth/user
描述: 获取当前用户信息
认证: 需要Bearer Token
响应:
- 200: {"id": 1, "username": "user123", "created_at": "2025-01-20T10:00:00"}
- 401: {"message": "未认证"}
"""

# ========================
```

```
# 2. 对话管理模块
# ========================

"""
POST /api/dialogues
描述: 创建新对话
认证: 需要Bearer Token
请求体:
{
    "title": "string, 可选, 默认'新对话'",
    "tags": ["string", "可选标签"]
}
响应:
- 201: {"id": 1, "title": "对话标题", "created_at": "2025-01-20T10:00:00"}
"""

"""
GET /api/dialogues
描述: 获取对话列表（分页）
认证: 需要Bearer Token
查询参数:
- page: integer, 页码, 默认1
- size: integer, 每页大小, 默认10
响应:
- 200: {
    "dialogues": [
        {"id": 1, "title": "标题", "created_at": "2025-01-20T10:00:00",
"message_count": 5}
    ],
    "total": 100,
    "page": 1,
    "size": 10
}
"""

"""
GET /api/dialogues/{id}
描述: 获取对话详情
认证: 需要Bearer Token
路径参数:
- id: integer, 对话ID
响应:
- 200: {
    "id": 1,
    "title": "标题",
    "created_at": "2025-01-20T10:00:00",
    "messages": [
```

```
        {"id": 1, "sender": "user", "content": "内容", "created_at": "2025-01-20T10:00:00"}
    ]
}
- 404: {"message": "对话不存在"}
"""

"""
GET /api/dialogues/{id}/messages
描述：获取对话消息（分页）
认证：需要Bearer Token
查询参数：
- page: integer, 页码，默认1
- size: integer, 每页大小，默认20
响应：
- 200: {
    "messages": [消息列表],
    "total": 50,
    "page": 1,
    "size": 20
}
"""

# ========================
# 3. 管仲AI核心模块
# ========================

"""
POST /api/guanzhong/chat
描述：与管仲AI进行实时对话
认证：可选（游客模式支持）
请求体：
{
    "message": "string, 必需，用户消息",
    "dialogue_id": "integer, 可选，关联对话ID"
}
响应：
- 200: {
    "ai_reply": "管仲AI的回复",
    "dialogue_id": "当前对话ID",
    "thought_process": {"economic": true, "political": false}
}
- 400: {"message": "请提供消息内容"}
- 500: {"message": "AI服务暂时不可用"}
"""

"""
```

```
POST /api/guanzhong/think
描述: 管仲AI深度思考（异步处理复杂问题）
认证: 需要Bearer Token
请求体:
{
    "question": "string, 必需, 复杂问题",
    "background": "string, 可选, 背景信息",
    "thinking_time": "integer, 可选, 思考时间（秒）"
}
响应:
- 202: {"task_id": "uuid", "status": "processing", "estimated_time": 30}
"""

"""
GET /api/guanzhong/topics
描述: 获取管仲思想主题列表
响应:
- 200: [
    {"id": 1, "title": "经济思想", "description": "仓廪实而知礼节...", "icon": "💰"}
]
"""

"""
GET /api/guanzhong/topics/{id}
描述: 获取主题详情
路径参数:
- id: integer, 主题ID
响应:
- 200: {
    "id": 1,
    "title": "经济思想",
    "description": "详细描述...",
    "key_concepts": ["概念1", "概念2"],
    "related_topics": [2, 3]
}
- 404: {"message": "主题不存在"}
"""

# ========================
# 4. 知识图谱模块
# ========================

"""
GET /api/knowledge/graph
描述: 获取管仲思想知识图谱
查询参数:
```

```
- depth: integer, 图谱深度，默认2
- root: string, 根节点，默认"管仲"
响应:
- 200: {
    "nodes": [
        {"id": "gz", "label": "管仲", "category": "人物", "size": 30}
    ],
    "edges": [
        {"source": "gz", "target": "jj", "label": "提出"}
    ]
}
"""

"""
GET /api/knowledge/concepts
描述: 获取概念列表（支持搜索）
查询参数:
- q: string, 搜索关键词
- category: string, 分类过滤
- page: integer, 页码
响应:
- 200: {
    "concepts": [
        {"id": 1, "name": "相地而衰征", "category": "经济政策", "description": "..."}
    ],
    "total": 100
}
"""

"""
GET /api/knowledge/concepts/{id}
描述: 获取概念详情
响应:
- 200: 概念详细信息
- 404: {"message": "概念不存在"}
"""

"""
GET /api/knowledge/relations
描述: 获取关系列表
查询参数:
- source: string, 源概念
- target: string, 目标概念
- type: string, 关系类型
响应:
- 200: 关系列表
```

```
"""

# ========================
# 5. 历史场景模块
# ========================

"""
GET /api/scenarios
描述: 获取历史场景列表
响应:
- 200: [
    {
        "id": 1,
        "title": "九合诸侯",
        "description": "场景描述...",
        "image_url": "图片URL",
        "difficulty": "easy|medium|hard",
        "estimated_time": 30
    }
]
"""

"""
GET /api/scenarios/{id}
描述: 获取场景详情
响应:
- 200: 场景详细信息
- 404: {"message": "场景不存在"}
"""

"""
POST /api/scenarios/{id}/enter
描述: 进入历史场景
认证: 需要Bearer Token
响应:
- 200: {"session_id": "场景会话ID", "instructions": "场景说明"}
"""

"""
POST /api/scenarios/{id}/action
描述: 在场景中执行动作
认证: 需要Bearer Token
请求体:
{
    "session_id": "string, 必需, 场景会话ID",
    "action": "string, 必需, 执行的动作",
    "parameters": "object, 可选, 动作参数"
```

```
}
响应:
- 200: {"result": "动作结果", "next_instructions": "下一步说明"}
"""

# ========================
# 6. 系统健康检查
# ========================

"""
GET /api/health
描述: 系统健康检查
响应:
- 200: {
    "status": "healthy",
    "timestamp": "2025-01-20T10:00:00",
    "version": "1.0.0",
    "services": {
        "database": "connected",
        "ai_service": "available"
    }
}
"""

"""
GET /api/version
描述: 获取API版本信息
响应:
- 200: {
    "name": "管仲AI后端API",
    "version": "1.0.0",
    "description": "管仲思想AI对话平台"
}
"""
```

app_backup.py

代码块

```
# backend/app.py
from flask import Flask
from flask_sqlalchemy import SQLAlchemy
from backend.models.user import db

app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///game.db'
```

```
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
db.init_app(app)

with app.app_context():
    db.create_all()

# 导入路由
from backend.routes.dialogue import bp as dialogue_bp
from backend.routes.game import bp as game_bp
from backend.routes.economy import bp as economy_bp

app.register_blueprint(dialogue_bp)
app.register_blueprint(game_bp)
app.register_blueprint(economy_bp)

if __name__ == '__main__':
    app.run(debug=True)
```

app.py

代码块

```
# 新增导入
import requests
import json
import os
import logging
from threading import Lock
from datetime import datetime
from flask import Flask, request, jsonify, send_from_directory
from flask_cors import CORS

# 正确初始化Flask应用
app = Flask(__name__)
CORS(app)  # 允许跨域请求

# 配置日志
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

# 豆包API配置（使用您提供的API Key）
app.config['DOUBAO_API_KEY'] = '573ab249-3130-4926-be5f-da98998863ef'
app.config['DOUBAO_API_URL'] = 'https://ark.cn-beijing.volces.com/api/v3/chat/completions'

# 线程安全锁
```

```python
dialogue_lock = Lock()

class DoubaoAIClient:
    @staticmethod
    def generate_reply(user_message):
        """调用豆包大模型生成管仲风格回复"""
        system_prompt = """
        你正在扮演春秋时期齐国名相管仲。请严格遵循以下角色设定：

        角色背景：
        - 春秋时期齐国名相，辅佐齐桓公成为春秋五霸之首
        - 思想核心：富民强国、礼义廉耻、尊王攘夷
        - 著名主张："仓廪实而知礼节，衣食足而知荣辱"
        - 政策：相地而衰征、官山海、轻重九府、四民分业

        回答要求：
        1. 使用文言文与现代汉语结合的风格（70%文言+30%现代）
        2. 体现管仲的治国智慧和哲学思想
        3. 适当引用《管子》中的经典语句
        4. 保持谦逊但权威的语气
        5. 回答要精炼，控制在100-200字
        6. 以"善哉！"、"吾闻之"等春秋时期用语开头

        示例回答：
        "善哉！治国之道，首在富民。仓廪实而知礼节，衣食足而知荣辱。民富则易治，民贫则难安。"
        """

        try:
            with dialogue_lock:
                headers = {
                    'Authorization': f'Bearer {app.config["DOUBAO_API_KEY"]}',
                    'Content-Type': 'application/json'
                }

                payload = {
                    "model": "doubao-seed-1-6-250615",  # 根据实际模型调整
                    "messages": [
                        {"role": "system", "content": system_prompt},
                        {"role": "user", "content": user_message}
                    ],
                    "temperature": 0.7,
                    "max_tokens": 500,
                    "stream": False
                }

```

```
                logger.info(f"发送请求到豆包API: {payload['messages'][1]['content'][:50]}...")

                response = requests.post(
                    app.config['DOUBAO_API_URL'],
                    headers=headers,
                    json=payload,
                    timeout=30
                )

                if response.status_code != 200:
                    error_msg = f"API请求失败: {response.status_code} - {response.text}"
                    logger.error(error_msg)
                    raise Exception(error_msg)

                result = response.json()
                logger.info("豆包API响应成功")
                return result['choices'][0]['message']['content']

        except Exception as e:
            logger.error(f"豆包API调用失败: {e}")
            return f"吾思有所阻，请稍后再试。错误: {str(e)}"

# 豆包对话API路由
@app.route('/api/doubao/chat', methods=['POST'])
def doubao_chat():
    """豆包大模型对话接口"""
    try:
        data = request.get_json()
        user_message = data.get('message', '').strip()

        if not user_message:
            return jsonify({"error": "消息内容不能为空"}), 400

        logger.info(f"收到用户消息: {user_message}")

        # 调用豆包API
        ai_reply = DoubaoAIClient.generate_reply(user_message)

        return jsonify({
            "success": True,
            "ai_reply": ai_reply,
            "model": "doubao",
            "api_key": app.config['DOUBAO_API_KEY'][:8] + "..." + app.config['DOUBAO_API_KEY'][-4:],
            "timestamp": datetime.now().isoformat()
```

```
        })

    except Exception as e:
        logger.error(f"对话处理错误: {e}")
        return jsonify({
            "success": False,
            "error": str(e),
            "ai_reply": "管仲沉思中，请稍候再试"
        }), 500

# 豆包API健康检查
@app.route('/api/doubao/health', methods=['GET'])
def doubao_health():
    """检查豆包API状态"""
    try:
        # 简单的测试请求
        test_reply = DoubaoAIClient.generate_reply("你好")
        return jsonify({
            "status": "healthy",
            "model": "doubao",
            "api_key_status": "有效",
            "response_time": "正常",
            "timestamp": datetime.now().isoformat()
        })
    except Exception as e:
        return jsonify({
            "status": "unhealthy",
            "error": str(e),
            "api_key_status": "无效",
            "timestamp": datetime.now().isoformat()
        }), 503

# 静态文件服务
@app.route('/')
def serve_index():
    """提供前端主页面"""
    return send_from_directory('../frontend/public', 'guanzhong_ai.html')

@app.route('/<path:path>')
def serve_static(path):
    """提供静态文件"""
    return send_from_directory('../frontend/public', path)

# 应用启动
if __name__ == '__main__':
    # 检查前端目录是否存在
    frontend_path = os.path.abspath('../frontend/public')
```

```
    if not os.path.exists(frontend_path):
        logger.warning(f"前端目录不存在: {frontend_path}")

    # 启动Flask应用
    app.run(
        debug=True,
        port=5000,
        host='0.0.0.0',
        threaded=True
    )
```

index.html

代码块

```
# 新增导入
import requests
import json
import os
import logging
from threading import Lock
from datetime import datetime
from flask import Flask, request, jsonify, send_from_directory
from flask_cors import CORS

# 正确初始化Flask应用
app = Flask(__name__)
CORS(app)  # 允许跨域请求

# 配置日志
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

# 豆包API配置（使用您提供的API Key）
app.config['DOUBAO_API_KEY'] = '573ab249-3130-4926-be5f-da98998863ef'
app.config['DOUBAO_API_URL'] = 'https://ark.cn-beijing.volces.com/api/v3/chat/completions'

# 线程安全锁
dialogue_lock = Lock()

class DoubaoAIClient:
    @staticmethod
    def generate_reply(user_message):
        """调用豆包大模型生成管仲风格回复"""
        system_prompt = """
```

```
        你正在扮演春秋时期齐国名相管仲。请严格遵循以下角色设定：

        角色背景：
        - 春秋时期齐国名相，辅佐齐桓公成为春秋五霸之首
        - 思想核心：富民强国、礼义廉耻、尊王攘夷
        - 著名主张："仓廪实而知礼节，衣食足而知荣辱"
        - 政策：相地而衰征、官山海、轻重九府、四民分业

        回答要求：
        1. 使用文言文与现代汉语结合的风格（70%文言+30%现代）
        2. 体现管仲的治国智慧和哲学思想
        3. 适当引用《管子》中的经典语句
        4. 保持谦逊但权威的语气
        5. 回答要精炼，控制在100-200字
        6. 以"善哉！"、"吾闻之"等春秋时期用语开头

        示例回答：
        "善哉！治国之道，首在富民。仓廪实而知礼节，衣食足而知荣辱。民富则易治，民贫则难安。"
        """

        try:
            with dialogue_lock:
                headers = {
                    'Authorization': f'Bearer {app.config["DOUBAO_API_KEY"]}',
                    'Content-Type': 'application/json'
                }

                payload = {
                    "model": "doubao-seed-1-6-250615",  # 根据实际模型调整
                    "messages": [
                        {"role": "system", "content": system_prompt},
                        {"role": "user", "content": user_message}
                    ],
                    "temperature": 0.7,
                    "max_tokens": 500,
                    "stream": False
                }

                logger.info(f"发送请求到豆包API: {payload['messages'][1]['content'][:50]}...")

                response = requests.post(
                    app.config['DOUBAO_API_URL'],
                    headers=headers,
                    json=payload,
                    timeout=30
```

```
                )

                if response.status_code != 200:
                    error_msg = f"API请求失败: {response.status_code} - {response.text}"
                    logger.error(error_msg)
                    raise Exception(error_msg)

                result = response.json()
                logger.info("豆包API响应成功")
                return result['choices'][0]['message']['content']

        except Exception as e:
            logger.error(f"豆包API调用失败: {e}")
            return f"吾思有所阻，请稍后再试。错误: {str(e)}"

# 豆包对话API路由
@app.route('/api/doubao/chat', methods=['POST'])
def doubao_chat():
    """豆包大模型对话接口"""
    try:
        data = request.get_json()
        user_message = data.get('message', '').strip()

        if not user_message:
            return jsonify({"error": "消息内容不能为空"}), 400

        logger.info(f"收到用户消息: {user_message}")

        # 调用豆包API
        ai_reply = DoubaoAIClient.generate_reply(user_message)

        return jsonify({
            "success": True,
            "ai_reply": ai_reply,
            "model": "doubao",
            "api_key": app.config['DOUBAO_API_KEY'][:8] + "..." + app.config['DOUBAO_API_KEY'][-4:],
            "timestamp": datetime.now().isoformat()
        })

    except Exception as e:
        logger.error(f"对话处理错误: {e}")
        return jsonify({
            "success": False,
            "error": str(e),
            "ai_reply": "管仲沉思中，请稍候再试"
```

```python
        }), 500

# 豆包API健康检查
@app.route('/api/doubao/health', methods=['GET'])
def doubao_health():
    """检查豆包API状态"""
    try:
        # 简单的测试请求
        test_reply = DoubaoAIClient.generate_reply("你好")
        return jsonify({
            "status": "healthy",
            "model": "doubao",
            "api_key_status": "有效",
            "response_time": "正常",
            "timestamp": datetime.now().isoformat()
        })
    except Exception as e:
        return jsonify({
            "status": "unhealthy",
            "error": str(e),
            "api_key_status": "无效",
            "timestamp": datetime.now().isoformat()
        }), 503

# 静态文件服务
@app.route('/')
def serve_index():
    """提供前端主页面"""
    return send_from_directory('../frontend/public', 'guanzhong_ai.html')

@app.route('/<path:path>')
def serve_static(path):
    """提供静态文件"""
    return send_from_directory('../frontend/public', path)

# 应用启动
if __name__ == '__main__':
    # 检查前端目录是否存在
    frontend_path = os.path.abspath('../frontend/public')
    if not os.path.exists(frontend_path):
        logger.warning(f"前端目录不存在: {frontend_path}")

    # 启动Flask应用
    app.run(
        debug=True,
        port=5000,
        host='0.0.0.0',
```

```
168             threaded=True
169         )
```